

장애물 제작 규칙 & 폴더 구조 가이드라인

엔진: Godot 4.6.1 / 언어: GDScript

작성일: 2026-05-27

새로운 장애물을 만들기 전에 반드시 읽을 것

1. 폴더 구조 규칙

전체 구조

```
프로젝트ver-2/
├── autoload/                                ← Autoload 전용 (SceneManager 등)
├── scripts/                                ← 모든 GDScript 파일
│   ├── player.gd
│   ├── platform.gd
│   ├── bullet.gd
│   ├── paint_mark.gd
│   ├── stage.gd
│   ├── main.gd
│   ├── color_defs.gd
│   └── obstacles/                          ← 장애물 스크립트 전용 폴더
│       ├── ghost_platform.gd
│       └── (새 장애물).gd
├── scenes/
│   ├── player/
│   │   └── Player.tscn
│   ├── platforms/
│   │   └── Platform.tscn
│   ├── obstacles/                          ← 장애물 씬 전용 폴더
│   │   ├── GhostPlatform.tscn
│   │   └── (새 장애물).tscn
│   ├── effects/
│   │   └── PaintMark.tscn
│   ├── bullet/
│   │   └── Bullet.tscn
│   ├── ui/                                ← UI 씬 전용
│   │   ├── MainMenu.tscn
│   │   └── StageSelect.tscn
│   └── world_1/
│       ├── stage_1/
│       │   └── stage_1.tscn
│       └── stage_2/
│           └── stage_2.tscn
└── assets/                                ← 모든 리소스 파일
    ├── textures/
    └── player/
```



폴더 네이밍 규칙

대상	규칙	예시
폴더	소문자 + 언더스코어	<code>obstacles/, world_1/</code>
썬 파일	PascalCase	<code>GhostPlatform.tscn</code>
스크립트	snake_case	<code>ghost_platform.gd</code>
텍스처	snake_case	<code>ghost_unpainted.png</code>
스테이지 썬	소문자	<code>stage_1.tscn</code>

⚠ 폴더 규칙 위반 시 생기는 문제

- 썬 파일과 스크립트가 뒤섞이면 나중에 30개 스테이지에서 파일 찾기 불가
- 텍스처가 `scenes/` 안에 있으면 Godot이 import 파일을 썬 폴더에 생성해 오염됨
- 스테이지 썬을 올바른 world 폴더에 넣지 않으면 SceneManager STAGES 딕셔너리 관리 불가

2. 물리 레이어 규칙

현재 레이어 목록 (project.godot)

레이어	이름	비트값	사용처
Layer 1	player	1	플레이어 본체
Layer 2	platform_black	2	검정 지형 충돌
Layer 3	platform_white	4	흰색 지형 충돌
Layer 4	platform_gray	8	회색 지형 충돌
Layer 5	bullet	16	총알
Layer 6	paint_detector	32	PaintMark JudgmentZone
Layer 7	kill_zone	64	DeathDetector
Layer 8	ghost_interact	128	투명 블록 감지

새 장애물 레이어 추가 시 규칙

1. project.godot의 [layer_names] 에 먼저 추가
2. 이름은 기능을 설명하는 snake_case
3. 이 문서의 레이어 목록을 반드시 업데이트
4. 총알이 감지해야 하면 bullet.tscn의 mask도 수정

⚠ 레이어 충돌 주의

새 장애물이 기존 레이어를 잘못 사용하면 의도치 않은 충돌이 발생함.

잘못된 예:

새 장애물이 layer_2(platform_black)를 사용

→ BLACK 플레이어가 장애물에 물리 충돌됨 (의도하지 않은 동작)

올바른 예:

새 레이어를 추가하거나, 역할이 동일하면 기존 레이어 재사용

3. 그룹(Group) 규칙

현재 사용 중인 그룹 목록

그룹명	등록 주체	읽는 주체	용도
paint_bodies	PaintMark, GhostPlatform	bullet.gd	총알 명중 시 update_color() 호출 대상
paint_marks	PaintMark의 JudgmentZone	player.gd	사망 판정 1순위
death_zones	Platform의 DeathDetector	player.gd	사망 판정 2순위

새 장애물의 그룹 등록 규칙

```
func _ready() -> void:
    # 총알이 색을 입힐 수 있어야 하면
    add_to_group("paint_bodies")

    # ColorSensor가 사망 판정을 해야 하면
    $DeathDetector.add_to_group("death_zones")
```

⚠ 그룹 오등록 시 발생하는 버그

실수	결과
death_zones 등록 안 함	플레이어가 반대색 장애물에 닿아도 사망 안 됨
paint_bodies 등록 안 함	총알이 장애물을 뚫고 지나감, 새 PaintMark가 엉뚱하게 생성됨
paint_marks 를 StaticBody2D에 등록	player.gd가 Area2D를 기대하는데 StaticBody2D가 들어와 오류

4. 사망 판정 연동 규칙

player.gd가 사망을 판단하는 방식

```
ColorSensor(Area2D) → get_overlapping_areas()
└─ 각 Area2D의 그룹 확인
    └─ "paint_marks" 그룹 → paint_color 프로퍼티 읽음
    └─ "death_zones" 그룹 → 부모의 color_state 프로퍼티 읽음
```

⚠ 프로퍼티명 반드시 확인

player.gd가 읽는 프로퍼티명과 장애물의 프로퍼티명이 **정확히 일치**해야 함.

```
# player.gd 가 읽는 방식 (get() 사용 - 없으면 null 반환)
var state = body.get("color_state")

# 장애물에서 반드시 같은 이름으로 선언
var color_state: int = ColorDefs.BLACK # ✅ 일치
var paint_color: int = ColorDefs.BLACK # ❌ 불일치 → 판정 안 됨
```

새 장애물 추가 전에 player.gd의 `_check_color_death()` 코드를 먼저 읽을 것

DeathDetector 설정 체크리스트

```
□ collision_layer = 64 (layer7 kill_zone)
□ collision_mask = 0
□ monitoring = false    ← DeathDetector가 다른 걸 감지할 필요 없음
□ monitorable = true    ← ColorSensor가 이걸 감지해야 함
                        (투명 상태이면 false로 시작)
```

5. 총알 상호작용 규칙

총알이 장애물에 닿는 흐름

```
bullet._on_body_entered(body)
└─ body가 "paint_bodies" 그룹?
    └─ YES → body.call_deferred("update_color", bullet_color)
    └─ NO  → PaintMark 새로 생성 + 총알 소멸
```

장애물이 총알에 맞으려면

1. 장애물의 `collision_layer`가 총알의 `collision_mask`에 포함돼야 함
2. `paint_bodies` 그룹에 등록돼야 함
3. `update_color(new_color: int)` 메서드가 반드시 있어야 함

update_color() 구현 필수 규칙

```
# ✅ 올바른 구현
func update_color(new_color: int) -> void:
    color_state = new_color
    call_deferred("_apply_color")    # collision 변경은 반드시 deferred

# ❌ 잘못된 구현 - physics flush 오류 발생
func update_color(new_color: int) -> void:
    color_state = new_color
    collision_layer = ...           # body_entered 콜백 내에서 직접 변경 금지
```

6. call_deferred 사용 규칙

반드시 call_deferred를 써야 하는 상황

상황	이유
body_entered / area_entered 콜백 내 add_child	물리 flush 중 씬 트리 변경 금지
body_entered 콜백 내 collision_layer 변경	물리 flush 중 충돌 상태 변경 금지
_ready() 내 change_scene_to_file	씬 트리 추가 중 씬 교체 금지

```
# ✅ 안전
func _on_body_entered(body):
    body.call_deferred("update_color", bullet_color)
    call_deferred("queue_free")

# ❌ 오류 발생
func _on_body_entered(body):
    body.update_color(bullet_color)    # 직접 호출
    queue_free()
```

7. ColorDefs 사용 규칙

색 상수는 반드시 ColorDefs 사용

```
# ✅ 올바른 방법
var color_state: int = ColorDefs.BLACK
if color_state == ColorDefs.WHITE:

# ❌ 잘못된 방법 - 파일마다 따로 정의하면 나중에 불일치 버그 발생
const BLACK: int = 0
const WHITE: int = 1
```

ColorDefs 값

```
# scripts/color_defs.gd
class_name ColorDefs
const BLACK: int = 0
const WHITE: int = 1
const GRAY: int = 2
```

8. @tool 사용 규칙

에디터에서 장애물 색상/상태를 미리 보려면 **@tool** 추가.
단, 아래 규칙을 반드시 지킬 것.

```
@tool
extends StaticBody2D

func _ready() -> void:
    if Engine.is_editor_hint():
        _apply_color() # 에디터에서 비주얼만 업데이트
        return        # 이후 런타임 로직 실행 금지

    # 런타임 로직
    add_to_group("paint_bodies")
    ...
```

⚠ **Engine.is_editor_hint()** 체크 없으면
에디터에서 그룹 등록, 시그널 연결 등이 실행돼 오류 발생

9. 씬 인스턴스 추가 규칙

add_child 전에 프로퍼티 설정

```
# ✅ 올바른 순서 - _ready() 호출 전에 설정 완료
var obstacle = OBSTACLE_SCENE.instantiate()
obstacle.color_state = ColorDefs.BLACK # 먼저 설정
get_tree().current_scene.add_child(obstacle)
obstacle.global_position = spawn_pos

# ❌ 잘못된 순서 - add_child 후 설정하면 _ready()가 먼저 실행돼 초기값 무시됨
var obstacle = OBSTACLE_SCENE.instantiate()
get_tree().current_scene.add_child(obstacle)
obstacle.color_state = ColorDefs.BLACK # 이미 _ready() 실행 후라 적용 안 될 수 있음
```

10. 새 장애물 제작 체크리스트

기획 단계

- 플레이어와의 물리 충돌이 있나? (없으면 Area2D 고려)
- 사망 판정이 필요한가?
- 총알에 맞는가? (맞으면 paint_bodies 그룹 + update_color 필요)
- 색상(BLACK/WHITE/GRAY)이 관련 있는가?
- 새 물리 레이어가 필요한가?
- 에디터에서 미리보기가 필요한가? (@tool)

파일 생성 단계

- scripts/obstacles/(이름).gd 생성
- scenes/obstacles/(이름).tscn 생성
- assets/textures/obstacles/(이름)_*.png 배치
- 이 문서 레이어 목록 업데이트 (레이어 추가 시)
- 가이드라인_장애물제작규칙.md 하단 장애물 목록 업데이트

코드 작성 단계

- ColorDefs 사용 (로컬 상수 금지)
- _ready() 최상단에 Engine.is_editor_hint() 체크 (@tool 사용 시)
- 그룹 등록 확인 (paint_bodies, death_zones)
- DeathDetector 설정 확인 (layer=64, monitorable)
- color_state 프로퍼티명 player.gd와 일치 확인
- update_color() 내부에서 call_deferred 사용
- collision_layer/mask 비트 계산 검사

테스트 단계

- 같은 색 플레이어 → 안전한가?
- 다른 색 플레이어 → 사망하는가?
- 총알 명중 → 색 변경되는가?
- 색 변경 후 사망 판정 재확인
- 에디터에서 @tool 동작 확인 (사용 시)
- F5 실행 시 오류 없는가?

11. 제작된 장애물 목록

이름	씬 경로	스크립트	설명
Platform	scenes/platforms/Platform.tscn	scripts/platform.gd	기본 색

이름	씬 경로	스크립트	설명
			상 발 판
GhostPlatform	scenes/obstacles/GhostPlatform.tscn	scripts/obstacles/ghost_platform.gd	투 명 → 칠 해 야 활 성 화

새 장애물 추가 시 이 목록에 반드시 추가할 것

이 문서는 프로젝트 진행에 따라 지속 업데이트 필요